

Wall-Following Robot

1. Project Overview

The **Wall-Following Robot** is an autonomous vehicle designed to navigate alongside a wall without colliding with it. Using an Arduino microcontroller and two ultrasonic sensors, the robot continuously scans its environment. It prioritizes frontal collision avoidance while maintaining a set distance from the wall on its right side, allowing it to successfully navigate mazes, hallways, or rooms.

2. Hardware Requirements

To build this robot, you will need the following components:

- **Microcontroller:** Arduino Uno
- **Sensors:** 2x HC-SR04 Ultrasonic Sensors (Front and Right)
- **Motor Driver:** L298N Motor Driver Module
- **Actuators:** 2x DC Gear Motors with Wheels
- **Chassis:** 2WD Robot Chassis kit (with a caster wheel)
- **Power Supply:** 2x 18650 Li-ion Batteries (7.4V total) with a battery holder
- **Miscellaneous:** Jumper wires, breadboard (optional), and double-sided tape.

3. Wiring & Connections

Ensure that the ESP32, L298N motor driver, and sensors share a **Common Ground (GND)** for the signals to work correctly.

Ultrasonic Sensors to Arduino

Sensor	HC-SR04 Pin	Arduino Pin
Front Sensor	VCC	5V
	Trig	Digital Pin 4
	Echo	Digital Pin 5
	GND	GND
Right Sensor	VCC	5V
	Trig	Digital Pin 6
	Echo	Digital Pin 7
	GND	GND

Motor Driver (L298N) to Arduino

L298N Pin	Arduino Pin	Purpose
IN1	Digital Pin 8	Left Motor Forward
IN2	Digital Pin 9	Left Motor Reverse
IN3	Digital Pin 10	Right Motor Forward
IN4	Digital Pin 11	Right Motor Reverse
12V / VCC	Battery (+)	Powers the motors
GND	Battery (-) & Arduino GND	Completes the circuit

Code:

```
// --- SENSOR PINS ---  
  
#define FRONT_TRIG 4  
  
#define FRONT_ECHO 5  
  
#define RIGHT_TRIG 6  
  
#define RIGHT_ECHO 7  
  
  
// --- MOTOR PINS (L298N) ---  
  
#define LEFT_MOTOR_IN1 8  
  
#define LEFT_MOTOR_IN2 9  
  
#define RIGHT_MOTOR_IN3 10  
  
#define RIGHT_MOTOR_IN4 11  
  
  
// --- DISTANCE SETTINGS (in cm) ---  
  
const int FRONT_STOP_DIST = 20; // If front wall is closer than 20cm, turn left  
  
const int IDEAL_WALL_DIST = 15; // The robot wants to stay 15cm away from the right wall  
  
const int ALLOWED_ERROR = 3; // Allows the robot to be between 12cm and 18cm without  
adjusting  
  
  
void setup() {  
  
    Serial.begin(9600);
```

```

// Setup Sensor Pins

pinMode(FRONT_TRIG, OUTPUT);
pinMode(FRONT_ECHO, INPUT);
pinMode(RIGHT_TRIG, OUTPUT);
pinMode(RIGHT_ECHO, INPUT);


// Setup Motor Pins

pinMode(LEFT_MOTOR_IN1, OUTPUT);
pinMode(LEFT_MOTOR_IN2, OUTPUT);
pinMode(RIGHT_MOTOR_IN3, OUTPUT);
pinMode(RIGHT_MOTOR_IN4, OUTPUT);
}


void loop() {
    // 1. Read distances from both sensors
    int frontDistance = getDistance(FRONT_TRIG, FRONT_ECHO);
    int rightDistance = getDistance(RIGHT_TRIG, RIGHT_ECHO);


    Serial.print("Front: "); Serial.print(frontDistance);
    Serial.print(" cm, Right: "); Serial.println(rightDistance);


    // 2. The Robot's Brain (Decision Making)


    if (frontDistance > 0 && frontDistance < FRONT_STOP_DIST) {
        // A wall is right in front of us! Turn hard left to avoid it.
        turnLeft();
    }
    else {
        // The path ahead is clear, so focus on following the right wall


        if (rightDistance < (IDEAL_WALL_DIST - ALLOWED_ERROR)) {

```

```

    // Too close to the wall! Steer slightly left away from it.
    turnSlightLeft();
}
else if (rightDistance > (IDEAL_WALL_DIST + ALLOWED_ERROR)) {
    // Too far from the wall! Steer slightly right towards it.
    turnSlightRight();
}
else {
    // We are at the perfect distance! Keep moving straight.
    moveForward();
}
}

// A small delay to keep the robot from reading sensors too fast and glitching
delay(50);
}

// Function to calculate distance from ultrasonic sensors
int getDistance(int trigPin, int echoPin) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH, 30000); // 30ms timeout
    if (duration == 0) return 999; // If no echo, assume path is clear

    return duration * 0.034 / 2; // Convert to cm
}

```

```
// Motor Control Functions
```

```
void moveForward() {
```

```
    digitalWrite(LEFT_MOTOR_IN1, HIGH);
```

```
    digitalWrite(LEFT_MOTOR_IN2, LOW);
```

```
    digitalWrite(RIGHT_MOTOR_IN3, HIGH);
```

```
    digitalWrite(RIGHT_MOTOR_IN4, LOW);
```

```
}
```

```
void turnLeft() {
```

```
    // Hard left: Left wheel backwards, Right wheel forwards
```

```
    digitalWrite(LEFT_MOTOR_IN1, LOW);
```

```
    digitalWrite(LEFT_MOTOR_IN2, HIGH);
```

```
    digitalWrite(RIGHT_MOTOR_IN3, HIGH);
```

```
    digitalWrite(RIGHT_MOTOR_IN4, LOW);
```

```
}
```

```
void turnSlightLeft() {
```

```
    // Soft left: Left wheel stops, Right wheel pushes forward
```

```
    digitalWrite(LEFT_MOTOR_IN1, LOW);
```

```
    digitalWrite(LEFT_MOTOR_IN2, LOW);
```

```
    digitalWrite(RIGHT_MOTOR_IN3, HIGH);
```

```
    digitalWrite(RIGHT_MOTOR_IN4, LOW);
```

```
}
```

```
void turnSlightRight() {
```

```
    // Soft right: Left wheel pushes forward, Right wheel stops
```

```
    digitalWrite(LEFT_MOTOR_IN1, HIGH);
```

```
    digitalWrite(LEFT_MOTOR_IN2, LOW);
```

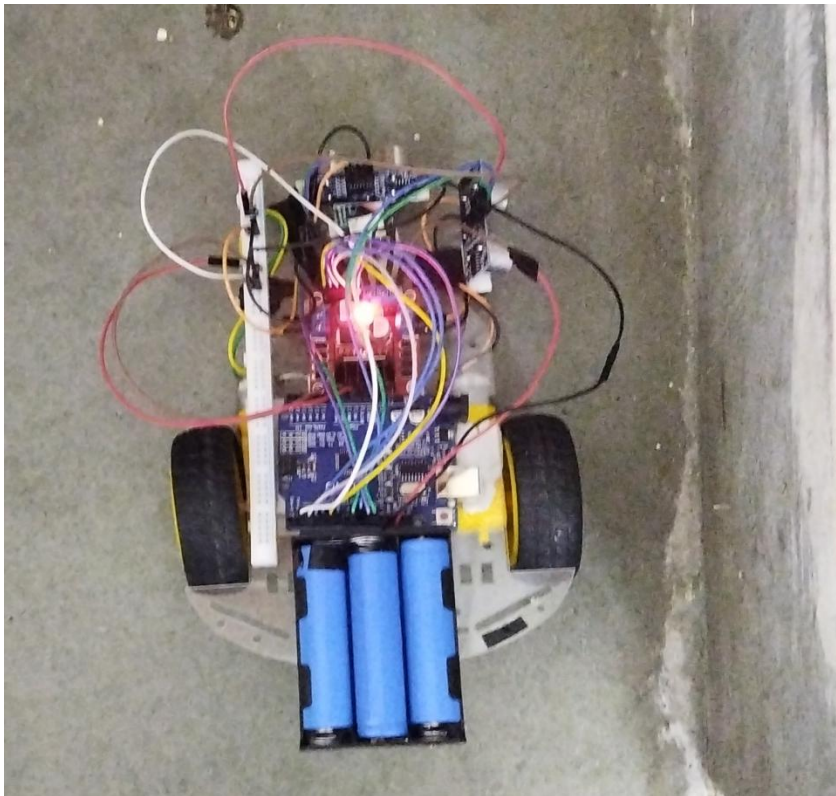
```
    digitalWrite(RIGHT_MOTOR_IN3, LOW);
```

```
    digitalWrite(RIGHT_MOTOR_IN4, LOW);
```

```
}
```

```
void stopMotors() {  
    digitalWrite(LEFT_MOTOR_IN1, LOW);  
    digitalWrite(LEFT_MOTOR_IN2, LOW);  
    digitalWrite(RIGHT_MOTOR_IN3, LOW);  
    digitalWrite(RIGHT_MOTOR_IN4, LOW);  
}
```

Output:



4. System Logic & Algorithm

The robot operates on a reactive, state-based algorithm that evaluates sensor data 20 times per second (every 50ms).

1. **Read Sensors:** Obtain distance in centimeters from both the Front and Right sensors.
2. **Obstacle Avoidance (Highest Priority):**
 - If Front Distance < 20 cm, a wall is directly ahead.
 - Action: Execute a hard left turn until the path is clear.
3. **Wall Tracking (Secondary Priority):**
 - If the front is clear, evaluate the right-side distance.

- **Too Close:** If Right Distance < 12 cm, stop the left motor to veer away from the wall (Slight Left).
- **Too Far:** If Right Distance > 18 cm, stop the right motor to steer back toward the wall (Slight Right).
- **Perfect Distance:** If Right Distance is between 12 cm and 18 cm, drive both motors forward.

5. Software Configuration

Development Environment

- **IDE:** Arduino IDE (v1.8.x or v2.x)
- **Board Package:** Arduino AVR Boards (Comes pre-installed with the Arduino IDE)
- **Board Selection:** "Arduino Uno" (or your specific Arduino model)
- **Baud Rate:** 9600 (for debugging in the Serial Monitor)

Key Variables to Tune

Depending on your motor speed and battery voltage, you may need to adjust these variables at the top of the code for optimal performance:

- **FRONT_STOP_DIST** (Default: 20): Increase this if the robot hits the front wall before it has time to turn.
- **IDEAL_WALL_DIST** (Default: 15): The target distance from the right wall.
- **ALLOWED_ERROR** (Default: 3): The buffer zone. Increase this if the robot wobbles left and right too violently.